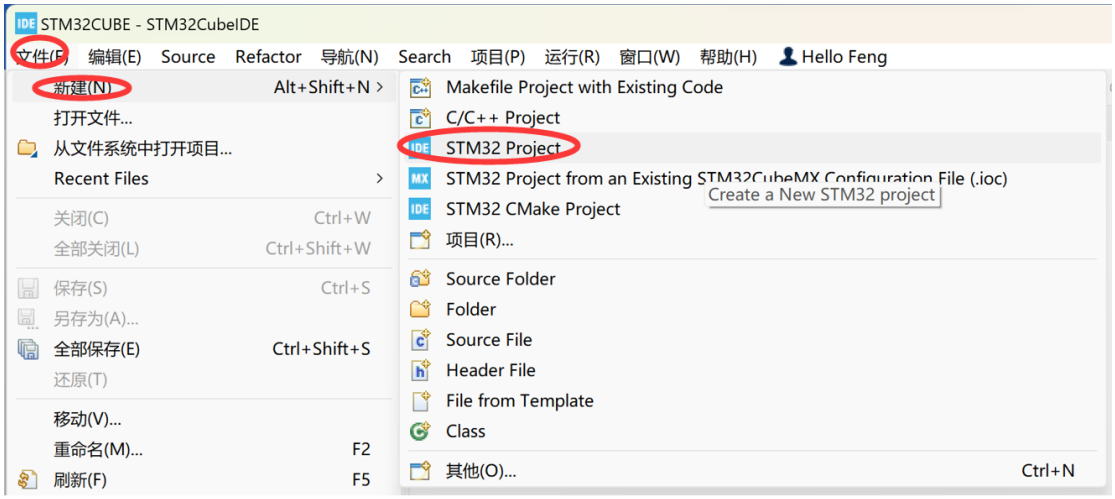
实践二 GPIO 控制和 UART 通信

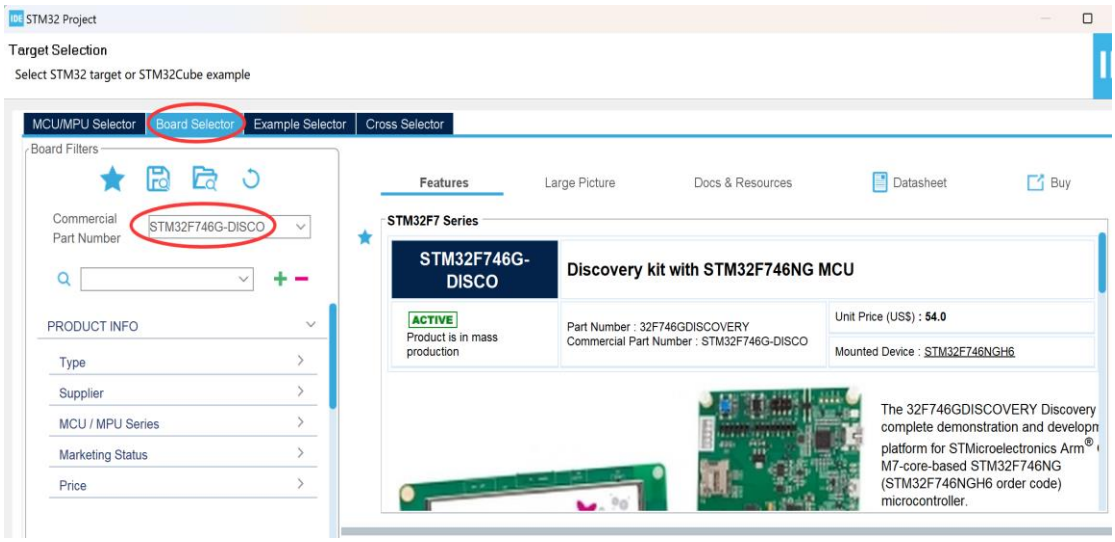
STM32F746-DISCO 开发板提供了 GPIO、I2C、ADC、PWM 和 USART 外围接口，可以根据项目设计需要进行选用。

一、 生成新项目

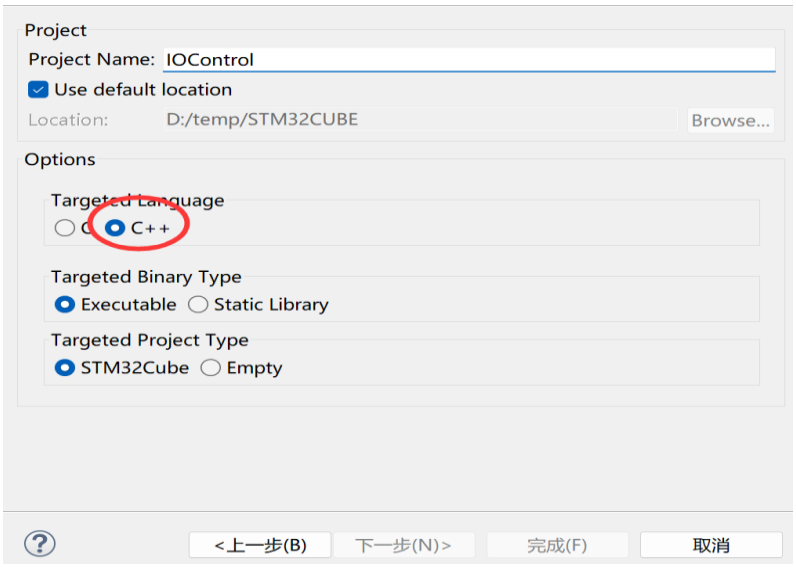
1. 在 STM32CubeIDE 中产生一个新的 STM32 项目



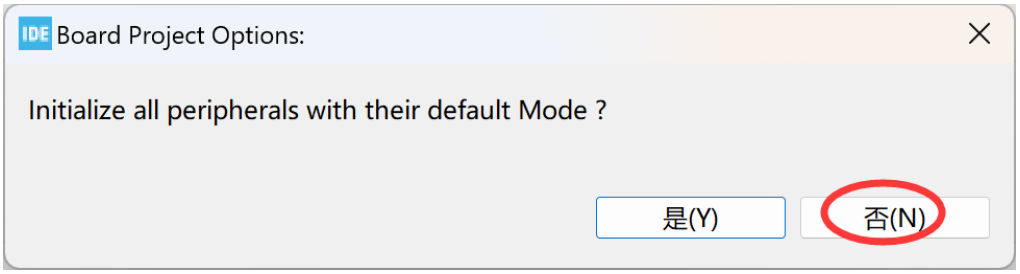
2. 选择 STM32F746G-DISCO 开发板。



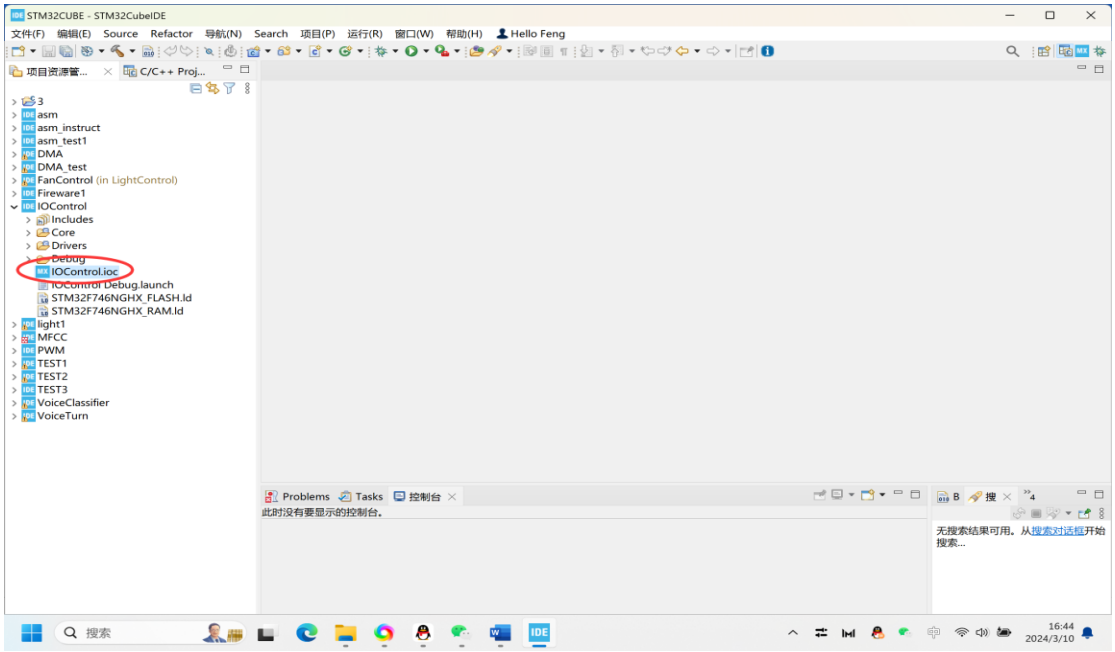
3. 配置目标为 C++项目，并点击“完成”



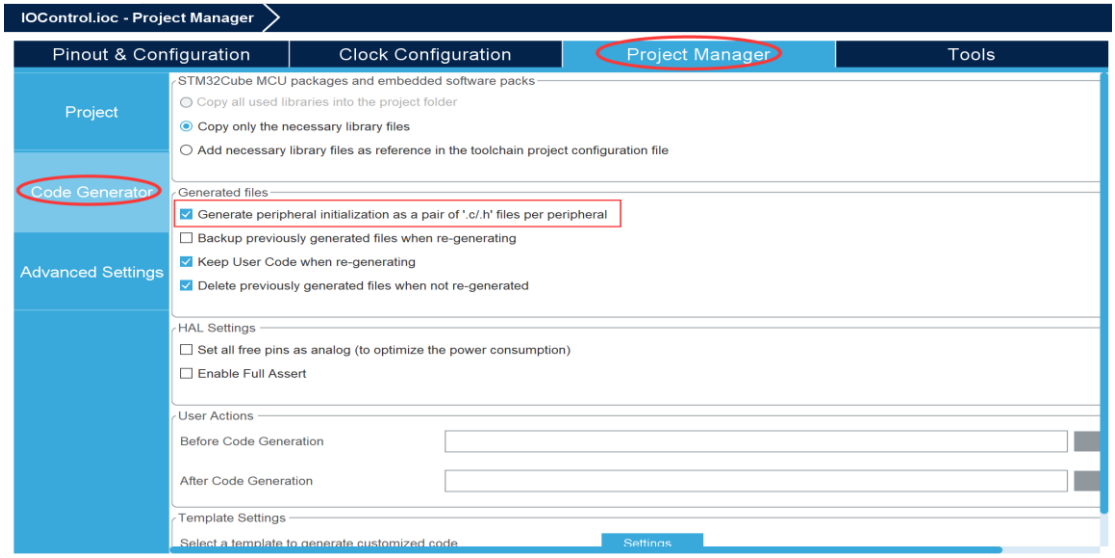
4. 在弹出框中选择“否”，以取消外设的默认 Mode



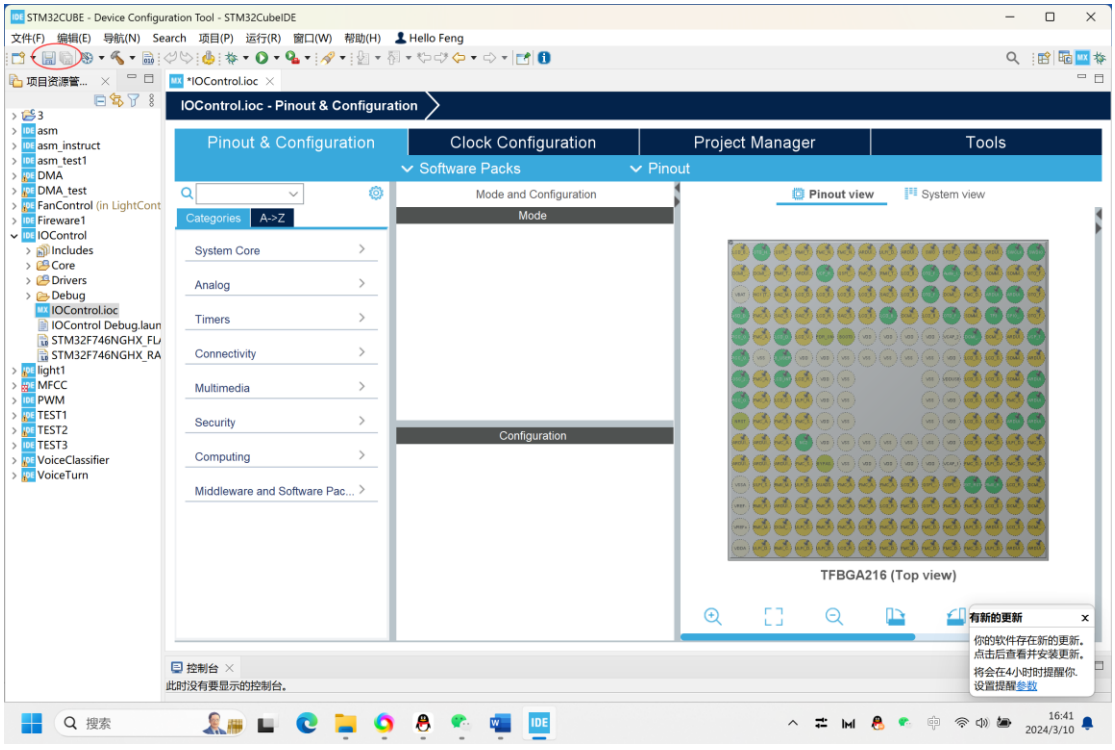
5. 在新生成的项目中，打开对应*.ioc 文件



6. 然后在 IOC 的“Project Manager->Code Generator”中设置为每个外设生成成对的“.c/.h”文件选项



7. 点击保存按钮，项目自动更新代码



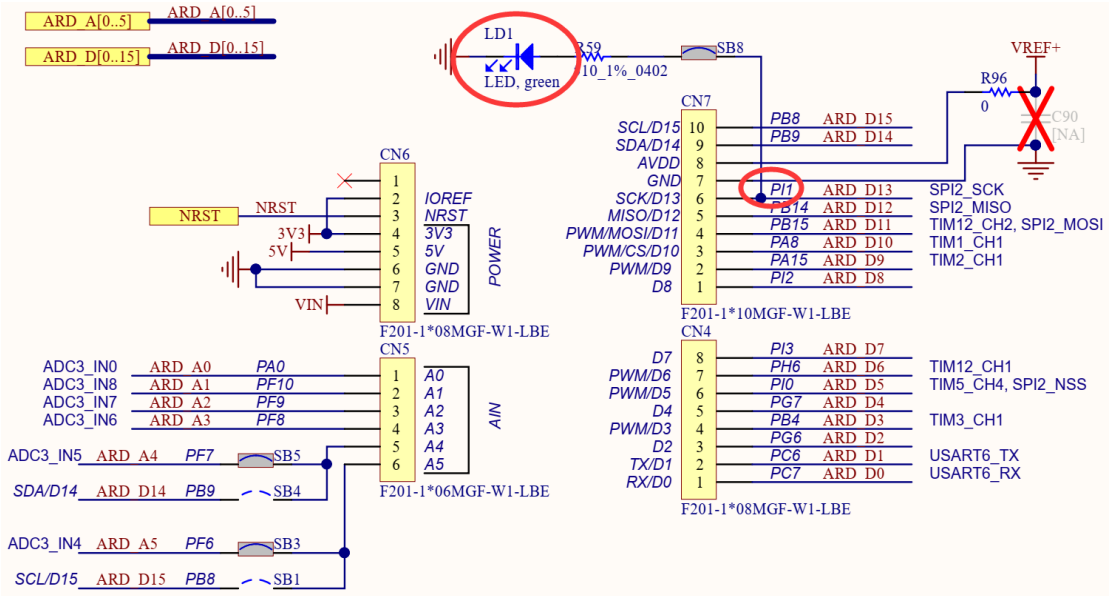
二、 GPIO 输出（LED）

1. GPIO 概述

GPIO（General Purpose Input/Output Port），即通用输入输出端口，就是芯片的一些引脚。作为输入端口时，可以通过它们读入引脚的状态——高电平或低电平；作为输出端口时，可以通过它们输出高电平或低电平来控制连接的外围设备。

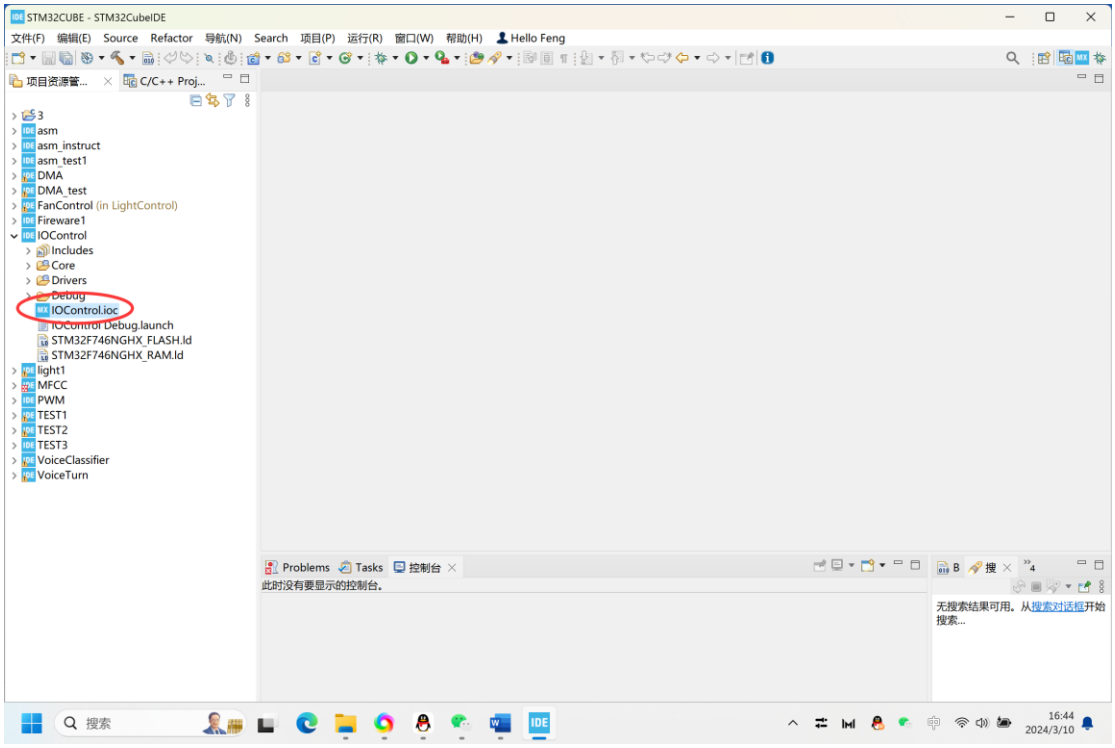
2. LED 接口

由前面下载的开发板原理图可知，开发板的 green LED 连接 GPIO 的 PI1 口。因此后面在 IOC 中对 PI1 进行配置。

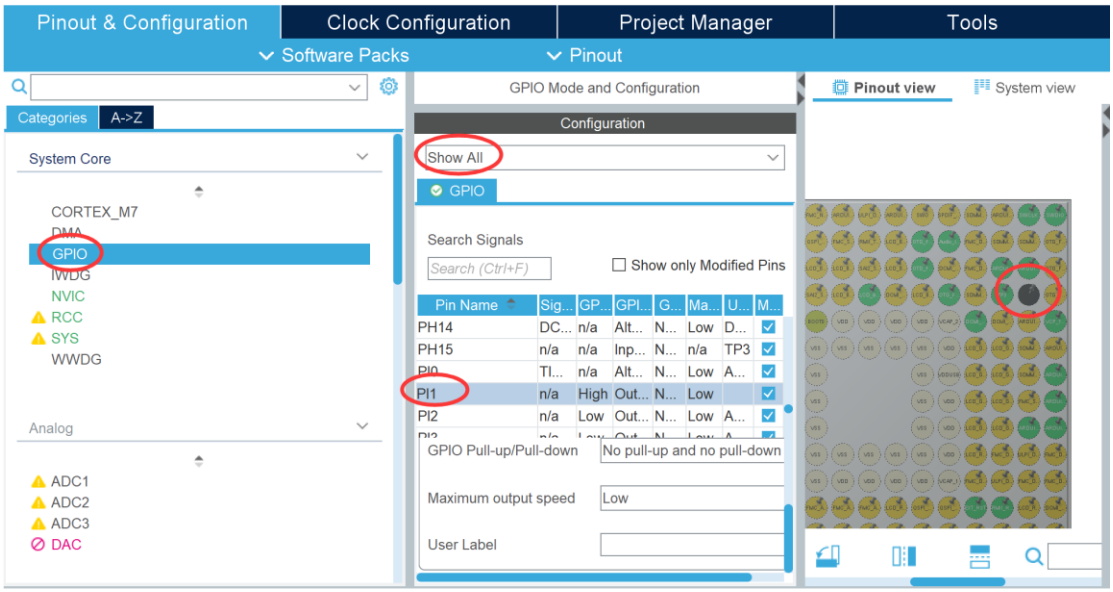


3. LED 配置

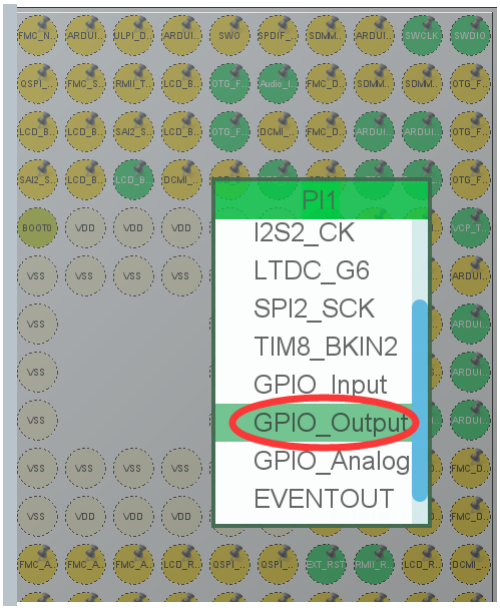
(1) 打开项目的*.ioc 文件



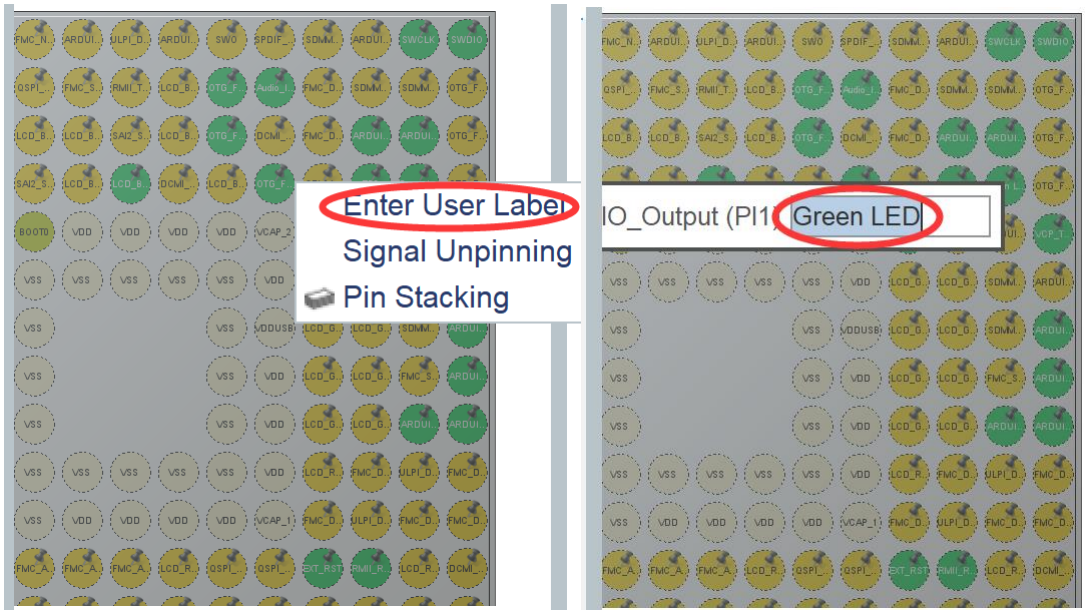
(2) 在“Pinout & Configuration”界面，从 GPIO 中找到 PI1 引脚



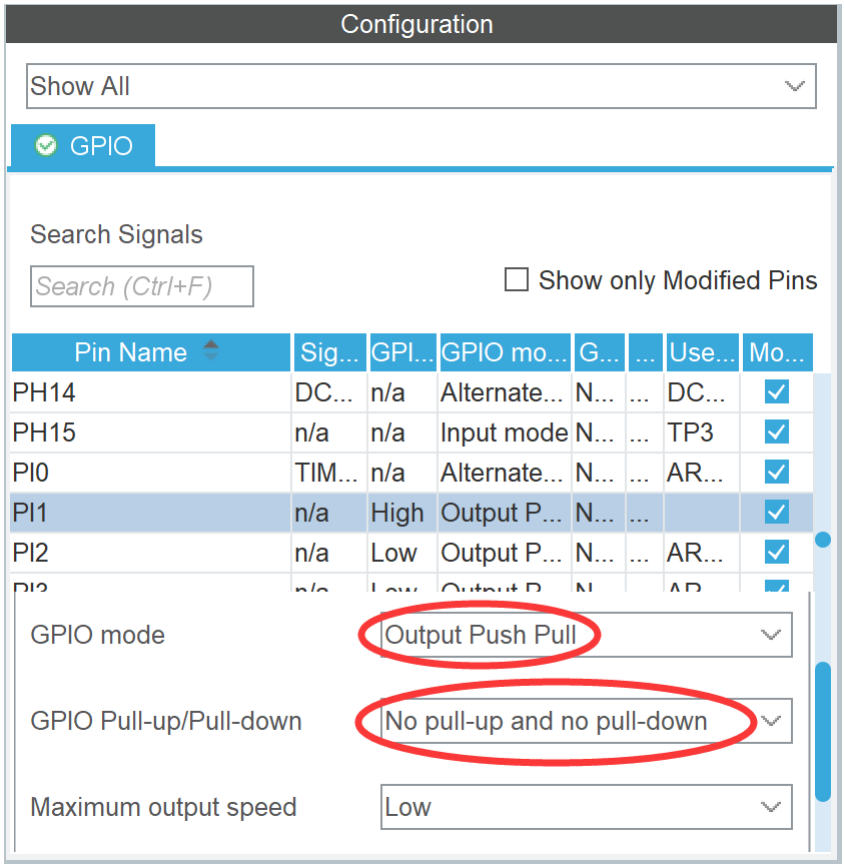
(3) 在“Pinout view”中对该引脚符号点击鼠标左键，在弹出菜单中选择将该引脚设置为“GPIO_Output”



(4) 在“Pinout view”中对该引脚符号点击右键，修改 User Label 为“Green LED”



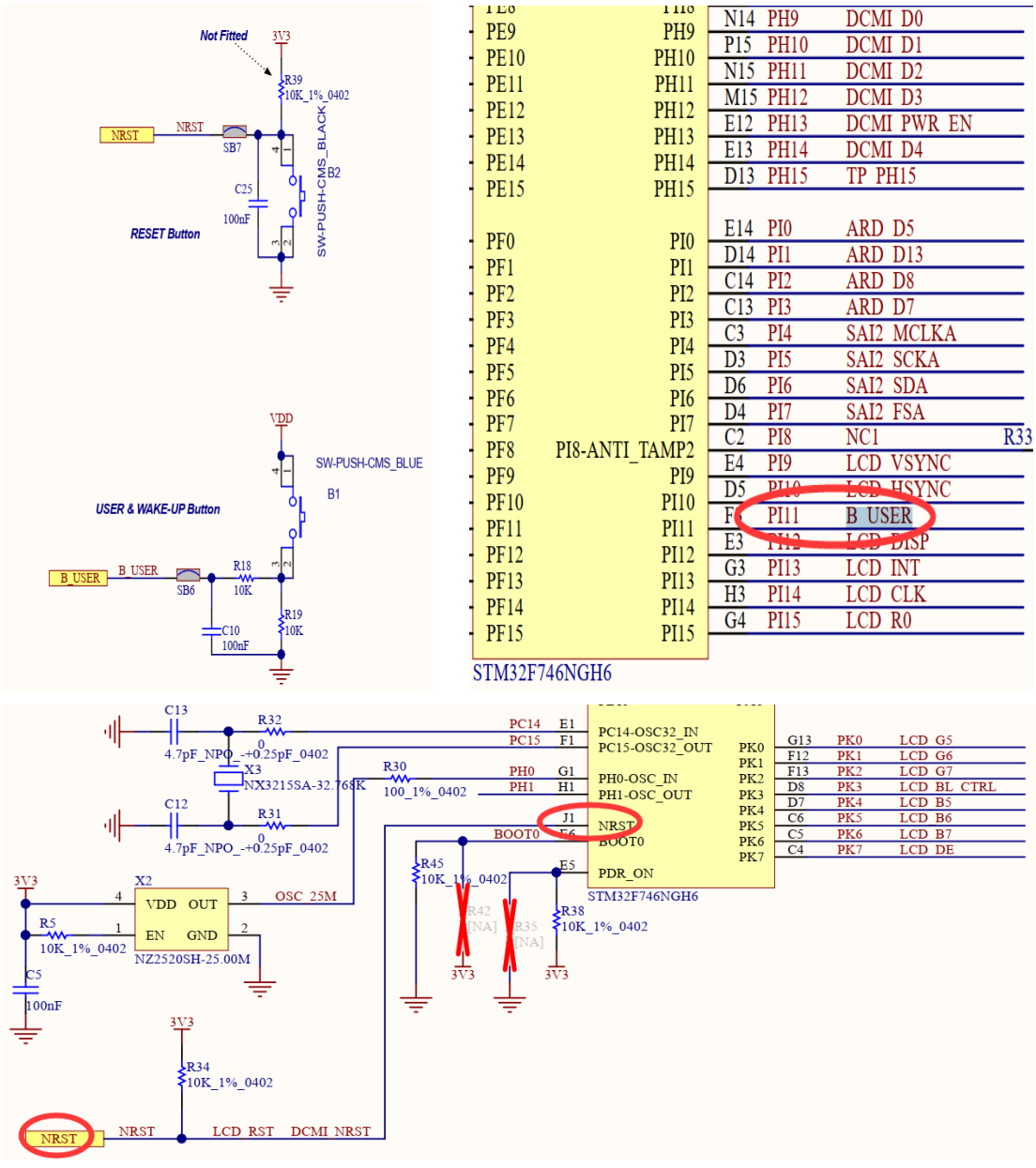
(5) 确认 PI1 口的“Mode”和上拉下拉电阻设置



三、 GPIO 输入（按键）

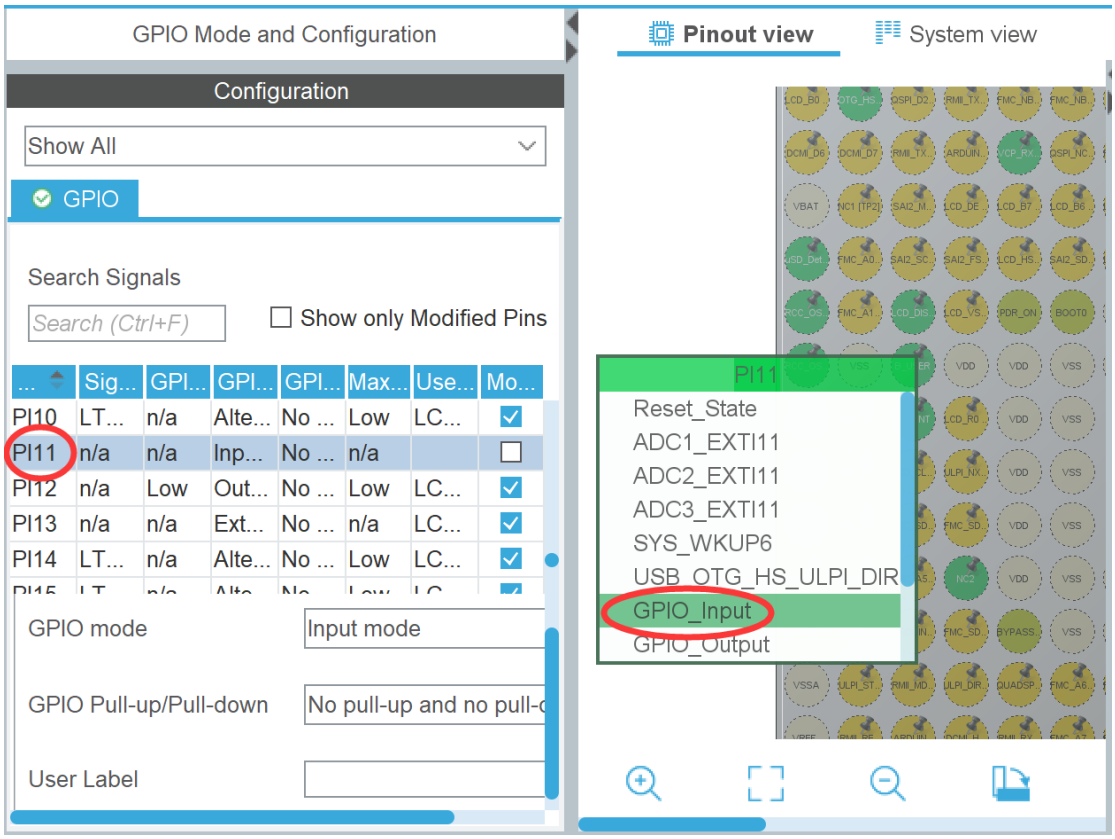
1. 用户 Button 接口

由开发板原理图可知，用户按键连接 PI11 口，下面对 PI11 口进行配置。



2. 用户 Button 配置

在 IOC 中将 PI11 设置为输入模式。

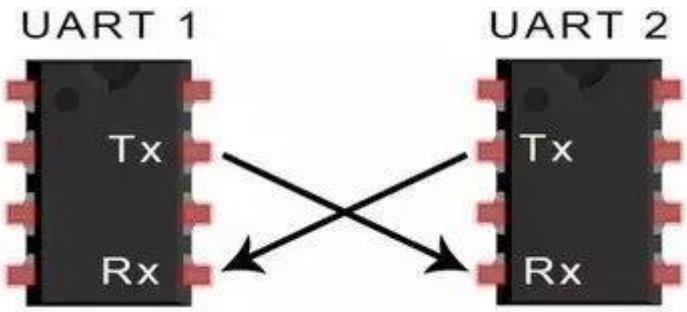


四、 UART 通信

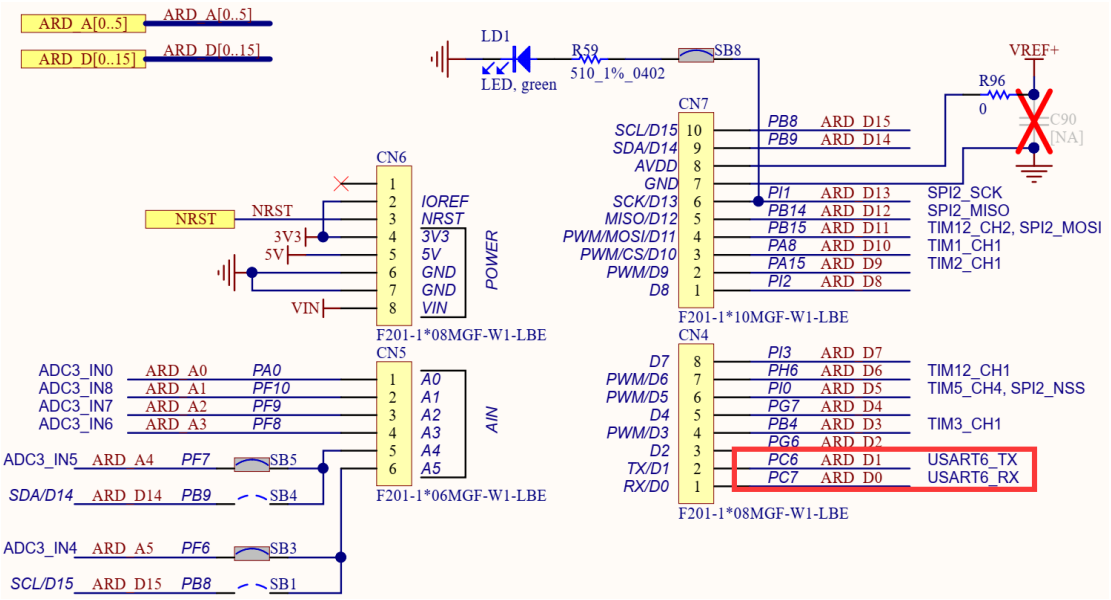
1. UART 通信简介

UART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）是一种常用的串行通信协议，广泛应用于各种嵌入式设备之间的通信。

UART 通信是一种**异步串行通信方式**，其基本原理是通过数据线上传输二进制数据位。UART 通信系统主要由发送端和接收端两部分组成，它们之间通过数据线进行数据传输。发送端将待发送的数据转换为并行信号，然后通过驱动电路将并行信号转换为串行信号，并通过发送电路将串行信号发送到数据线上。接收端则通过接收电路将数据线上的信号还原为并行信号，再通过解码电路将并行信号转换为原始数据位。

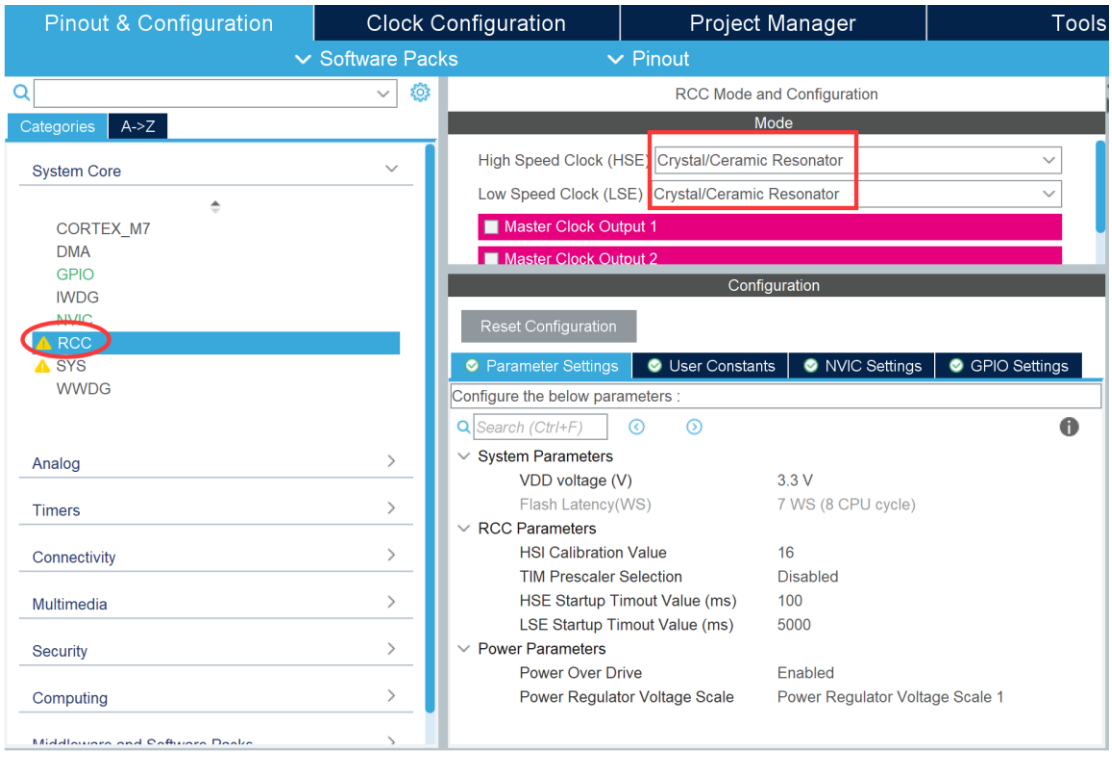


2. USART6 引脚连接

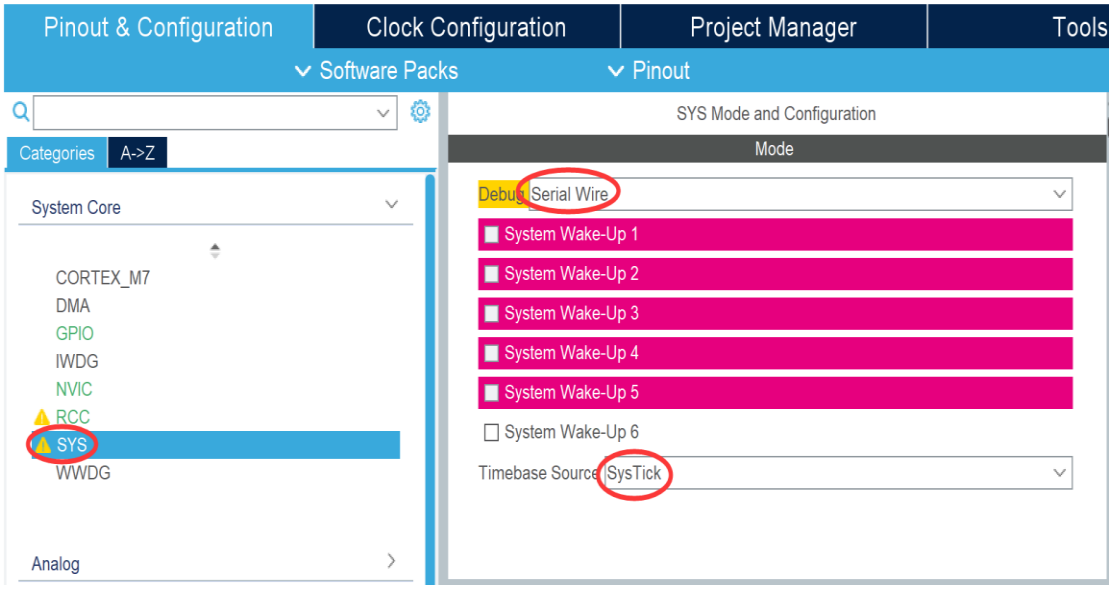


3. UART 接口配置

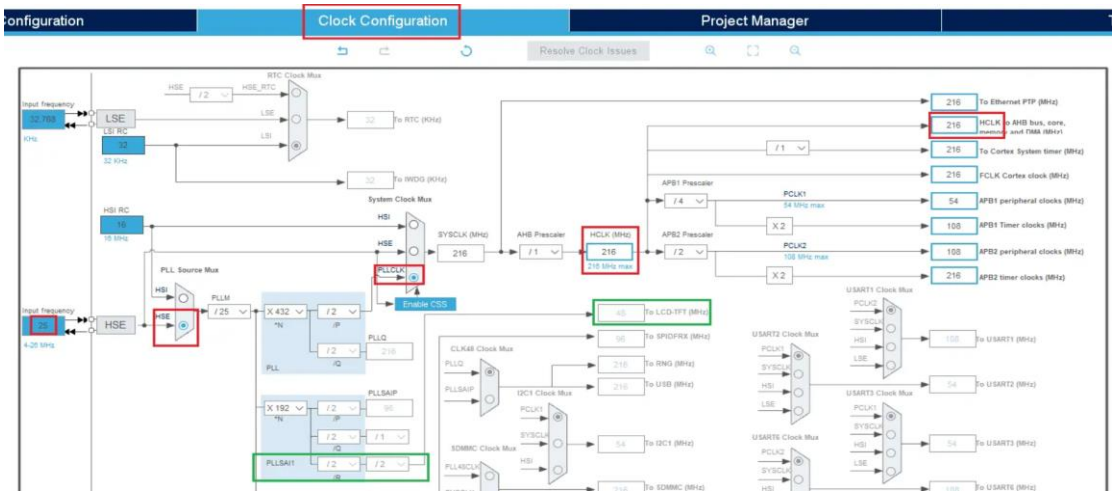
(1) RCC（Reset Clock Control，复位和时钟控制器）配置外部晶振



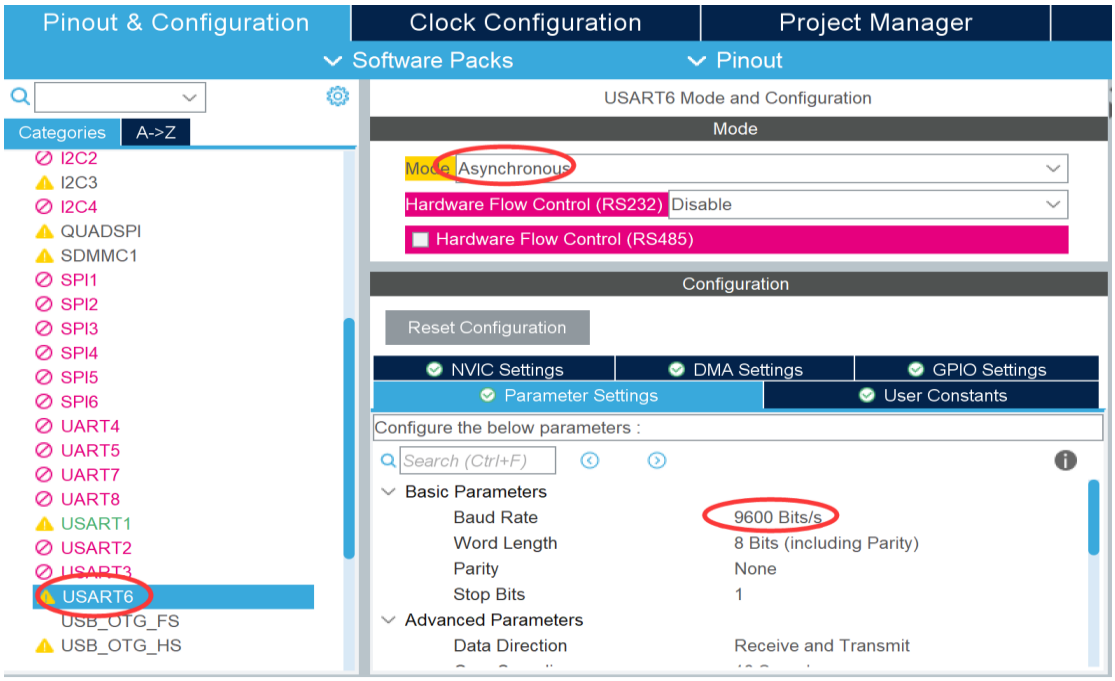
(2) 配置调试模式（ST-LINK）



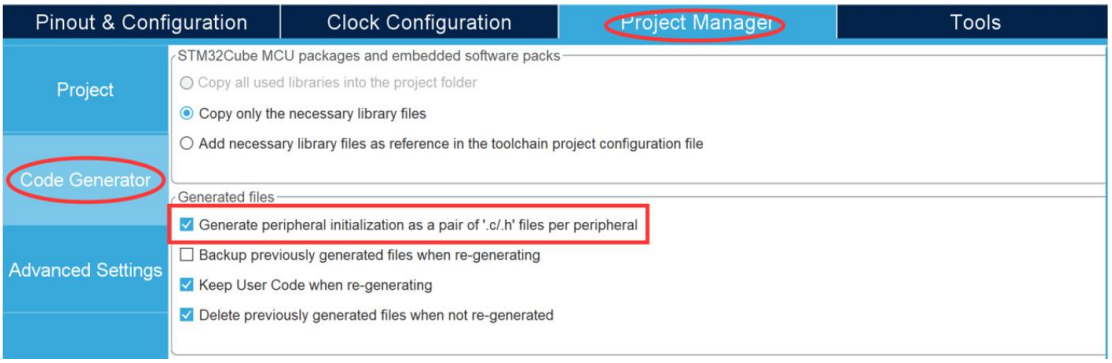
(3) 配置主时钟时钟



(4) 配置 USART6



(6) 保存“.ioc”文件，生成对应的代码，注意先选择如下选项。



五、 轮询、中断和 DMA 数据传输

1. 轮询代码

(1) 在 main.c 中添加头文件

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
```

(2) 添加字符串定义

```
/* USER CODE BEGIN 1 */
char button_up[]= "Button up!\n";
char button_down[]= "Button down!\n";
/* USER CODE END 1 */
```

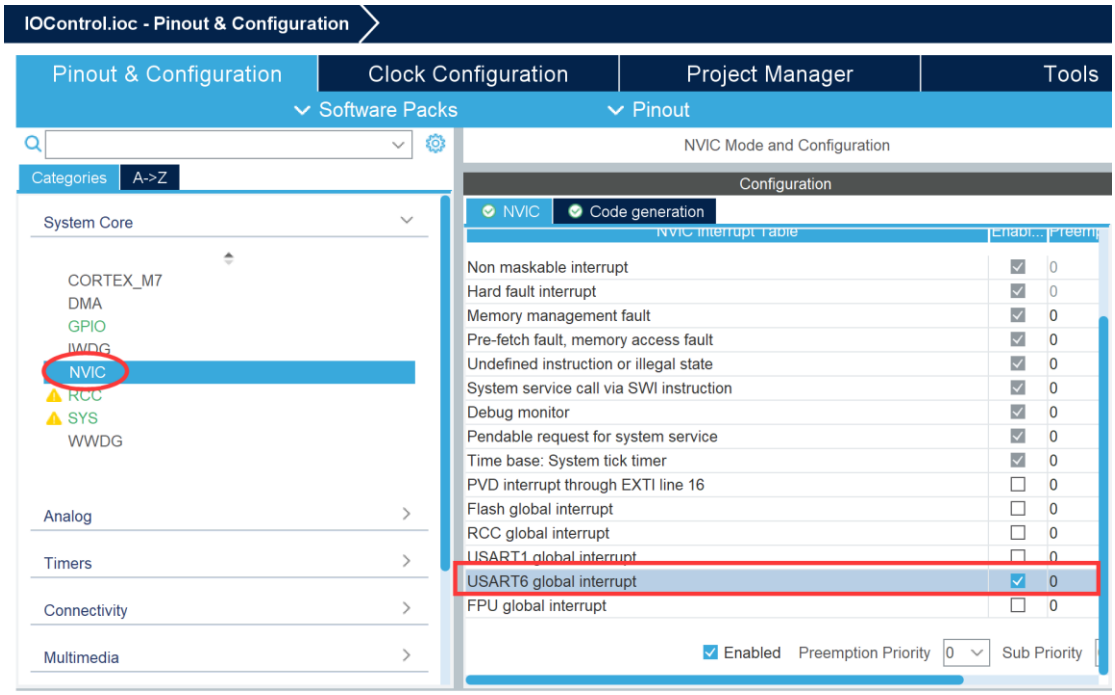
(3) 添加 while 循环里的代码内容

```
while (1)
{
    /* USER CODE END WHILE */
    if (HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_11) == GPIO_PIN_RESET) {
           HAL_UART_Transmit(&huart6, (uint8_t*)button_up,
                             strlen(button_up), 0xffffff);
           HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET);
       }
       else
       {
           HAL_UART_Transmit(&huart6, (uint8_t*)button_down,
                             strlen(button_down), 0xffffff);
           HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET);
       }
       HAL_Delay(500);
    /* USER CODE BEGIN 3 */
}
```

2. 中断代码

参考视频链接：

https://www.bilibili.com/video/BV1bc411J7Tv?vd_source=8d628cca437d3f6b85e95a32fb0de0f2



(1) 添加变量

```
/* USER CODE BEGIN PV */
uint8_t receiveData[2];
GPIO_PinState state;
/* USER CODE END PV */
```

(2) 添加中断回调函数

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    HAL_UART_Transmit_IT(&huart6,receiveData,2);
    state = GPIO_PIN_SET;
    if(receiveData[1] == '0'){
       state = GPIO_PIN_RESET;
    }
    if(receiveData[0] == 'G'){
```

```
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, state);
    }
    HAL_UART_Receive_IT(&huart6,receiveData,2);
}
/* USER CODE END 0 */
```

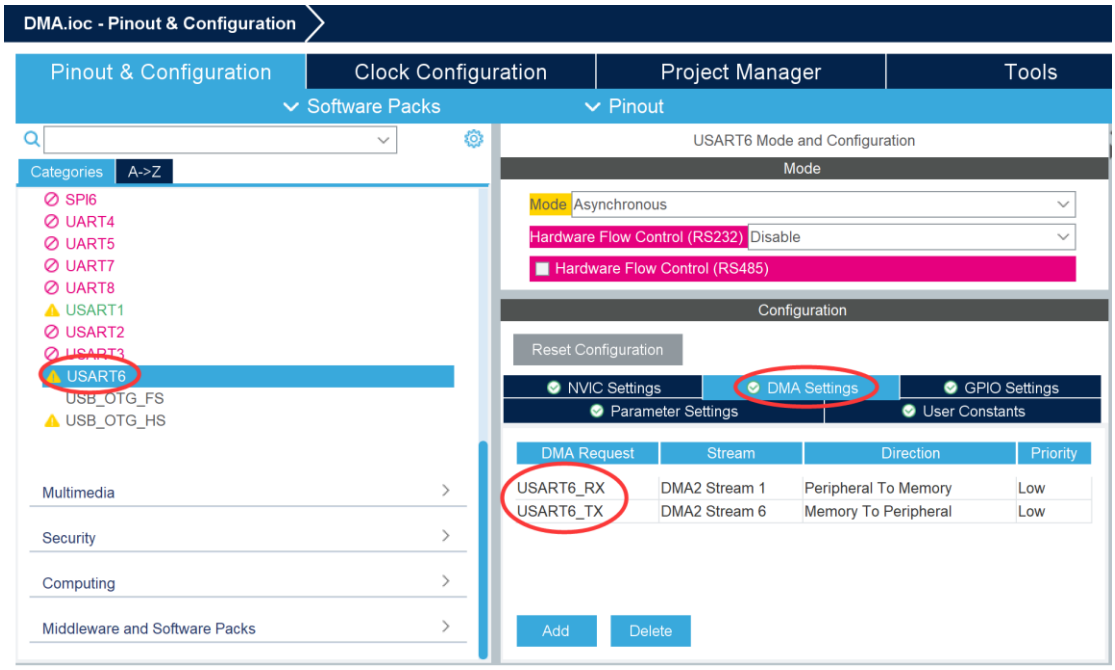
(3) 添加中断接收串口命令

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart6,receiveData,2);
/* USER CODE END 2 */
```

3. DMA 代码

参考视频链接：

https://www.bilibili.com/video/BV1do4y1F7wt?vd_source=8d628cca437d3f6b85e95a32fb0de0f2



(1) 添加变量

```
/* USER CODE BEGIN PV */
uint8_t receiveData[2];
GPIO_PinState state;
/* USER CODE END PV */
```

(2) 添加中断回调函数

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    HAL_UART_Transmit_DMA(&huart6,receiveData,2);
    state = GPIO_PIN_SET;
    if(receiveData[1] == '0'){
       state = GPIO_PIN_RESET;
    }
    if(receiveData[0] == 'G'){
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, state);
    }
    HAL_UART_Receive_DMA(&huart6,receiveData,2);
}
/* USER CODE END 0 */
```

(3) 添加 DMA 接收串口命令

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_DMA(&huart6,receiveData,2);
/* USER CODE END 2 */
```

4. DMA 接收任意数据

(1) 添加变量

```
/* USER CODE BEGIN PV */
uint8_t receiveData[50];
GPIO_PinState state;
/* USER CODE END PV */
```

(2) 添加中断回调函数

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    HAL_UART_Transmit_DMA(&huart6,receiveData,2);
    state = GPIO_PIN_SET;
    if(receiveData[1] == '0'){
       state = GPIO_PIN_RESET;
    }
```

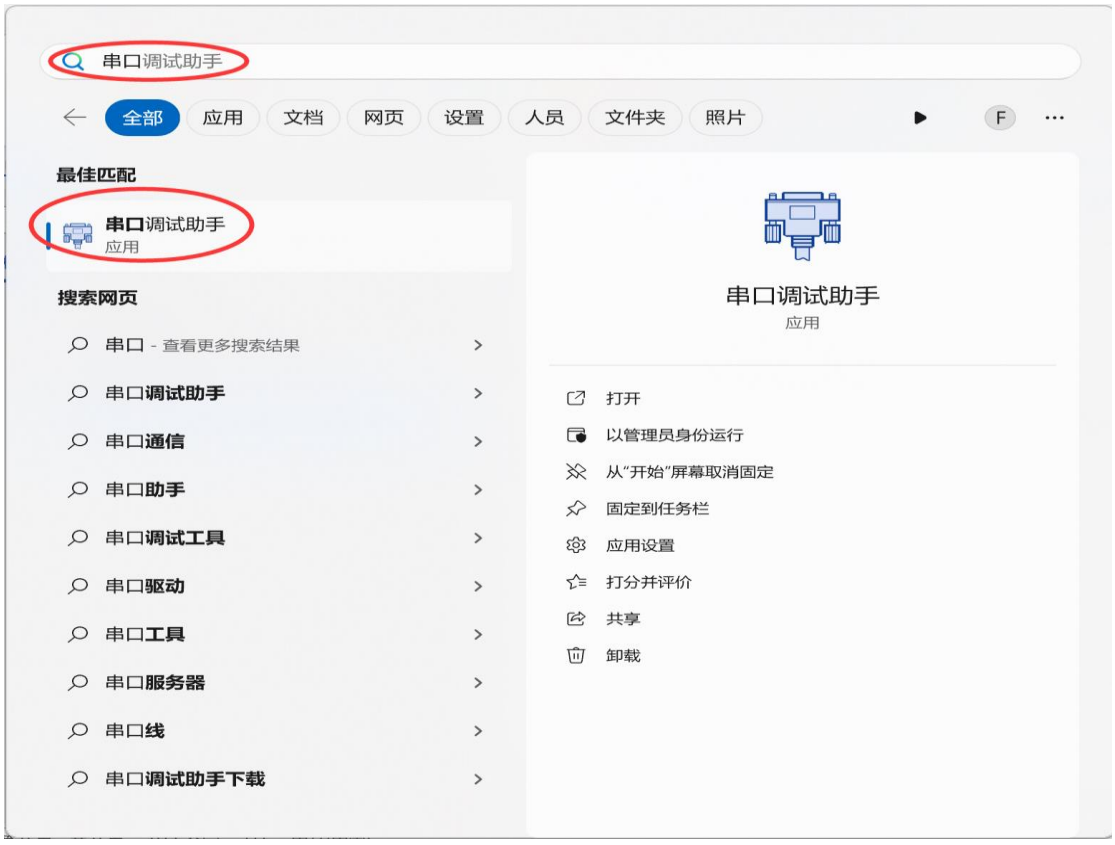
```
    if(receiveData[0] == 'G'){
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, state);
    }
    HAL_UART_Receive_DMA(&huart6,receiveData,2);
}
/* USER CODE END 0 */
```

(3) 添加中断接收串口命令

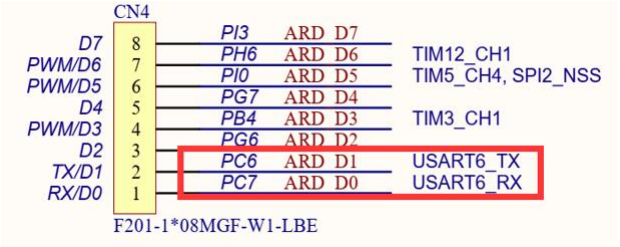
```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_DMA(&huart6,receiveData,2);
/* USER CODE END 2 */
```

六、 通过串口监测数据输出

(1) 在 Windows11 应用中查找“串口调试助手”并安装



(2) 连接串口模块发送和接收两根线到开发板 CN4 口的 1、2，同时再连接地线。另一端插到电脑的 USB 口。然后在 STM32CubeIDE 中运行测试程序。



(3) 在 PC 机端设置串口调试助手的波特率为 9600，按下“打开串口”键，查看代码运行的输出。

